

Milestone 3 - PDF for grading.

Goohs

D1 – See Milestone3_D1.pdf diagram for protocol.

D2 - Prove your design will operate correctly during multiple overlapping transactions. [10 pts]

Proof by Contradiction:

Assume k processes (where $k \geq 2$) are currently in the critical section, protected by a working semaphore S initialized to 1.

Since all k processes are in the critical section, each must have successfully executed $WAIT(S)$. Each successful $WAIT(S)$ operation decrements S by 1:

After 1st process: $S = 1 - 1 = 0$

After 2nd process: $S = 0 - 1 = -1$

After 3rd process: $S = -1 - 1 = -2$

...

After k th process: $S = 1 - k$

Since $k \geq 2$, we have $S = 1 - k \leq 1 - 2 = -1$.

Contradiction: After the first process executes $WAIT(S)$, the semaphore value becomes 0. When the second process attempts $WAIT(S)$, it finds $S=0 < 1$, so it cannot proceed and must be placed in the semaphore's waiting queue. However, our assumption states this second process (and all subsequent processes) are already in the critical section, which directly contradicts the fact that they should be blocked in the waiting queue.

Therefore, our assumption is false, and at most one process can be in the critical section at any time.

D3 - Describe, in reasonable detail, why your group chose this particular design. [5 pts]

We chose this design to protect against race conditions, ensure data integrity via a mutual exclusion (semaphore), and more generally maintain a simplistic and reliable

synchronization functionality. When dealing with a shared resource, we run into the issue of multiple users attempting to modify the balance at the same time. Our mutex implementation prevents this by creating a lock that allows only one ATM in the critical section at a time. Although speed/efficiency are idealistic in system design, we felt for an ATM system, correctness meets the specifications of the bank president at a much higher precedence. Lastly, the design benefits throughout scaling of the ATM/bank network as it uses the built-in queuing mechanism of the semaphore preventing starvation of a process, and also easy error recovery (a sys admin could just reset the semaphore which would allow the counter to return to 1 and the next process in the queue to move into the CS.)